TECHNICAL REFERENCE

AIRFLOW 3.X COMPATIBLE

Apache Airflow

Workflow Orchestration Platform

■ COMPLETE PRACTITIONER REFERENCE · CHAPTERS 1–3

Operators, Sensors & Hooks

A production-grade deep-dive into Airflow's execution primitives. Every operator, sensor, and hook — with parameters, when/why guidance, and real production code examples with full syntax highlighting.

18	9	10	3.x
OPERATORS	SENSORS	HOOKS	AIRFLOW

- CH 1 — Operators
- CH 2 — Sensors
- CH 3 — Hooks
- BashOperator
- PythonOperator
- BranchPythonOperator
- Deferrable & Async
- Dynamic Task Mapping
- FileSensor
- ExternalTaskSensor
- HttpHook
- SSHHook & SFTPHook

Debajyoti Mukhopadhyay

Solution Architect · Data Platform Engineering

March 2026

airflow.apache.org

CONTENTS

Table of Contents

CH 1 Operators

1.1 BashOperator

1.2 PythonOperator

1.3 BranchPythonOperator

1.4 ShortCircuitOperator

1.5 PythonVirtualenvOperator

1.6 BranchPythonVirtualenvOperator

1.7 ExternalPythonOperator

1.8 BranchExternalPythonOperator

1.9 BranchDateTimeOperator

1.10 BranchDayOfWeekOperator

1.11 LatestOnlyOperator

1.12 TriggerDagRunOperator

1.13 EmptyOperator

1.14 BaseOperator

1.15 EmailOperator

1.16 Custom Operators

1.17 Deferrable Operators & Async Execution

1.18 MappedOperator — Dynamic Task Mapping

CH 2 Sensors

2.1 FileSensor / FileSystemSensor

2.2 ExternalTaskSensor

2.3 TimeDeltaSensor

2.4 TimeSensor

2.5 DateTimeSensor

2.6 BashSensor

2.7 PythonSensor

2.8 SqlSensor

2.9 HttpSensor

CH 3 Hooks

3.1 BaseHook

3.2 FSHook

3.3 SubprocessHook

3.4 KubernetesHook

3.5 DockerHook

3.6 HttpHook

3.7 FTPHook

3.8 SFTPHook

3.9 SSHHook

3.10 EmailHook

CHAPTER 1 OF 3

CH 1 Operators — Complete Reference

Operators are the atomic units of work in Airflow — executing shell commands, calling Python, sending email, triggering other DAGs. Each operator encapsulates exactly one task. This chapter covers all 18 operators with parameters, when/why guidance, and full production code examples.

18 operators covered in this chapter

Shell 1.1 BashOperator

```
from airflow.operators.bash import BashOperator
```

Executes a Bash shell command or script on the worker node. The command is Jinja-rendered before execution, so you can inject {{ ds }} , environment variables, and connection URIs at runtime. Exit code 0 = success; any non-zero exit = failure (unless mapped to skip_exit_code). Stdout is optionally pushed to XCom via do_xcom_push . This is the simplest way to run scripts, CLI tools, dbt, or shell utilities from within Airflow.

PARAMETER	TYPE / DEFAULT	DESCRIPTION
bash_command	Shell command string or path to .sh script to execute	
env	Dict of env vars to inject; inherits parent env if None	
append_env	If True, merge env dict with the inherited environment	
skip_exit_code	Exit code that causes SKIPPED state instead of failure	
do_xcom_push	Push last stdout line as XCom return_value (default True)	
cwd	Working directory for the command	

PYTHON

```python
from airflow.operators.bash import BashOperator

# ── Example 1: Date-injected command ──────────────────────────────
run_export = BashOperator(
    task_id="run_daily_export",
    bash_command="python /opt/etl/export.py --date={{ ds }} --env=prod",
    dag=dag,
)

# ── Example 2: Multi-line script with env vars, skip on no-data ──────
check_and_export = BashOperator(
    task_id="check_and_export",
    bash_command="""
        set -euo pipefail
        echo "Processing date: $RUN_DATE"
        ROW_COUNT=$(psql $DB_URL -t -c "SELECT COUNT(*) FROM events WHERE date='$RUN_DATE'")
        echo "Rows found: $ROW_COUNT"
        if [ "$ROW_COUNT" -eq "0" ]; then
            echo "No data — skipping" && exit 99
        fi
        python /opt/etl/export.py --date=$RUN_DATE
        echo "$ROW_COUNT"   # Last line pushed to XCom as return_value
    """,
    env={"RUN_DATE": "{{ ds }}", "DB_URL": "{{ conn.postgres_prod.get_uri() }}"},
    append_env=True,
    skip_exit_code=99,
    do_xcom_push=True,
```

```
    dag=dag,
)
```

Python 1.2 PythonOperator

```
from airflow.operators.python import PythonOperator
```

> *The most commonly used operator. Executes any Python callable (function, lambda, or method) in the worker process. The return value is automatically pushed to XCom as return_value . With provide_context=True (default in Airflow 2.x), the full task context — execution_date , ti , dag , ds , etc. — is injected via **kwargs . Use op_kwargs for static parameters and templates_dict for Jinja-rendered values.*

PYTHON

```python
from airflow.operators.python import PythonOperator
from airflow.decorators import dag, task
from datetime import datetime

# ── Classic style ──────────────────────────────────────────────────────
def fetch_sales(ds: str, execution_date, ti, **context) -> dict:
    """
    ds             → "YYYY-MM-DD" string
    execution_date → pendulum.DateTime
    ti             → TaskInstance — use ti.xcom_pull() to read upstream
    """
    upstream = ti.xcom_pull(task_ids="validate_source", key="return_value")
    print(f"Processing {ds}, upstream: {upstream}")
    return {"rows": 9500, "date": ds}   # auto-pushed to XCom["return_value"]

fetch = PythonOperator(
    task_id="fetch_sales",
    python_callable=fetch_sales,
    op_kwargs={"region": "APAC"},
    dag=dag,
)

# ── @task decorator (recommended in Airflow 2.x) ──────────────────────
@dag(schedule_interval="@daily", start_date=datetime(2024, 1, 1), catchup=False)
def sales_pipeline():

    @task
    def extract(ds: str) -> dict:
        return {"rows": 9500, "date": ds}

    @task
    def transform(raw: dict) -> dict:
        return {"cleaned": int(raw["rows"] * 0.97)}

    @task
    def load(data: dict) -> None:
        print(f'Loaded {data["cleaned"]} rows')

    raw = extract()
    cleaned = transform(raw)
    load(cleaned)

dag_instance = sales_pipeline()
```

Branching 1.3 BranchPythonOperator

```
from airflow.operators.python import BranchPythonOperator
```

> *A specialised PythonOperator for conditional branching. The callable must return a task_id string or list of task_ids . Airflow executes only those downstream tasks and SKIPs all other branches. After branching, use EmptyOperator with trigger_rule=NONE_FAILED_MIN_ONE_SUCCESS to cleanly re-join parallel branches. Never use ALL_SUCCESS as the convergence trigger rule — skipped branches would cause the join to fail.*

PYTHON

```python
from airflow.operators.python import BranchPythonOperator
from airflow.operators.empty import EmptyOperator
from airflow.utils.trigger_rule import TriggerRule
from airflow.models import Variable

def decide_pipeline_path(**context) -> str:
    env       = Variable.get("environment", default_var="dev")
    row_count = context["ti"].xcom_pull(task_ids="count_rows")

    if env == "prod" and row_count > 10_000:
        return "run_full_load"
    elif env == "prod" and row_count > [full_load, incr_load, sample_load, notify_dev] >> converge
```

Flow Control 1.4 ShortCircuitOperator

```
from airflow.operators.python import ShortCircuitOperator
```

> *Evaluates a Python callable that returns a boolean. True → pipeline continues normally. False → ALL downstream tasks are marked SKIPPED . Cleaner than BranchPythonOperator when there is only one downstream path. Use ignore_downstream_trigger_rules=True to force-skip even tasks with ALL_DONE trigger rules.*

PYTHON

```python
from airflow.operators.python import ShortCircuitOperator

def is_business_day(execution_date, **context) -> bool:
    """Return False on weekends — skip the entire downstream pipeline."""
    return execution_date.weekday()  bool:
    """True only when GCS files have arrived for this date."""
    from airflow.providers.google.cloud.hooks.gcs import GCSHook
    hook  = GCSHook(gcp_conn_id="google_cloud_default")
    files = hook.list(bucket_name="data-lake", prefix=f"raw/{ds}/")
    return len(files) > 0   # False → skip everything downstream

guard_weekday = ShortCircuitOperator(
    task_id="guard_weekday",
    python_callable=is_business_day,
    provide_context=True,
)

guard_data = ShortCircuitOperator(
    task_id="guard_data_arrived",
    python_callable=has_new_data,
    provide_context=True,
    ignore_downstream_trigger_rules=True,
)
```

```
process = EmptyOperator(task_id="process_data")
notify  = EmptyOperator(task_id="send_report")

guard_weekday >> guard_data >> process >> notify
```

Isolated 1.5 PythonVirtualenvOperator

```
from airflow.operators.python import PythonVirtualenvOperator
```

Runs a Python callable inside a freshly created virtualenv, completely isolated from Airflow's own Python environment. Ideal when a task requires package versions that conflict with Airflow dependencies (e.g., a specific torch , scipy , or great_expectations version). Use venv_cache_path to cache the virtualenv and avoid re-creating it on every run. Arguments and return values are serialised via cloudpickle .

PYTHON

```python
from airflow.operators.python import PythonVirtualenvOperator

def run_ml_inference(model_path: str, data_path: str) -> dict:
    """Runs in isolated venv — specific torch version, no conflict with Airflow."""
    import torch
    import pandas as pd

    model = torch.load(model_path)
    df    = pd.read_parquet(data_path)
    preds = model(torch.tensor(df.values, dtype=torch.float32))
    return {"predictions": preds.tolist(), "count": len(df)}

ml_task = PythonVirtualenvOperator(
    task_id="run_ml_inference",
    python_callable=run_ml_inference,
    requirements=[
        "torch==2.1.0",
        "pandas==2.0.3",
        "pyarrow==14.0.0",
    ],
    system_site_packages=False,        # Fully isolated — no worker packages
    python_version="3.11",
    venv_cache_path="/tmp/airflow_venvs",   # Reuse across runs
    op_kwargs={
        "model_path": "gs://ml-bucket/models/v3.pt",
        "data_path":  "gs://ml-bucket/features/{{ ds }}.parquet",
    },
    dag=dag,
)
```

Isolated Branch 1.6 BranchPythonVirtualenvOperator

```
from airflow.operators.python import BranchPythonVirtualenvOperator
```

Combines virtualenv isolation with branching logic. Use when your branching decision itself requires third-party libraries unavailable in the Airflow environment. The callable must return a task_id string (or list) indicating which branch to follow.

PYTHON

```python
from airflow.operators.python import BranchPythonVirtualenvOperator

def classify_data_quality(gcs_path: str) -> str:
    """Runs in isolated venv with great_expectations. Returns task_id to follow."""
    import great_expectations as ge

    context = ge.data_context.DataContext()
    result  = context.run_checkpoint(checkpoint_name="daily_quality")
    score   = result.statistics["successful_expectations_percent"]

    if score >= 0.95:
        return "load_to_prod"
    elif score >= 0.80:
        return "load_to_quarantine"
    else:
        return "trigger_data_incident"

branch_quality = BranchPythonVirtualenvOperator(
    task_id="branch_on_quality",
    python_callable=classify_data_quality,
    requirements=["great-expectations==0.18.0"],
    op_kwargs={"gcs_path": "gs://bucket/{{ ds }}/data.parquet"},
    dag=dag,
)
```

External Env

1.7 ExternalPythonOperator

```python
from airflow.operators.python import ExternalPythonOperator
```

> *Executes a Python callable using an existing, pre-installed Python interpreter on the filesystem — faster than PythonVirtualenvOperator since no environment creation step is needed. Useful for conda environments, legacy Python installs, or shared ML environments already present on every worker node.*

PYTHON

```python
from airflow.operators.python import ExternalPythonOperator

def run_spark_script(date: str) -> None:
    """Uses the PySpark Python install at /opt/spark/bin/python."""
    import pyspark
    from pyspark.sql import SparkSession

    spark = SparkSession.builder.appName("DailyAgg").getOrCreate()
    df    = spark.read.parquet(f"gs://bucket/raw/{date}/")
    agg   = df.groupBy("customer_id").sum("amount")
    agg.write.mode("overwrite").parquet(f"gs://bucket/agg/{date}/")
    spark.stop()

spark_task = ExternalPythonOperator(
    task_id="spark_aggregation",
    python="/opt/spark/bin/python",   # Path to external interpreter
    python_callable=run_spark_script,
    op_kwargs={"date": "{{ ds }}"},
    dag=dag,
)
```

External Branch

1.8 BranchExternalPythonOperator

```
from airflow.operators.python import BranchExternalPythonOperator
```

The branching counterpart of ExternalPythonOperator. Runs branching logic using a specific external Python interpreter and returns a task_id or list of task_ids. Use when your entire task group depends on a shared external Python environment (e.g., a shared ML conda env) and that same environment contains the logic needed to make the routing decision.

PYTHON

```python
from airflow.operators.python import BranchExternalPythonOperator

def choose_model_version() -> str:
    """Uses /opt/ml-env/bin/python which has scikit-learn 1.3."""
    import joblib

    current = joblib.load("/opt/models/current_version.pkl")
    score   = current.score_threshold

    if score > 0.92:
        return "use_current_model"
    else:
        return "retrain_model"

branch_model = BranchExternalPythonOperator(
    task_id="branch_model_selection",
    python="/opt/ml-env/bin/python",
    python_callable=choose_model_version,
    dag=dag,
)
```

Time Branch

1.9 BranchDateTimeOperator

```
from airflow.operators.datetime import BranchDateTimeOperator
```

Routes the DAG to different branches based on whether the execution falls within a specified datetime window. No Python callable required — branching is purely time-configured via target_lower and target_upper . Perfect for business-hours logic, maintenance windows, and time-gated feature flags.

PYTHON

```python
from airflow.operators.datetime import BranchDateTimeOperator
from datetime import time

branch_time = BranchDateTimeOperator(
    task_id="check_business_hours",
    follow_task_ids_if_true=["run_realtime_load"],    # 09:00 – 17:00
    follow_task_ids_if_false=["run_batch_load"],      # outside window
    target_lower=time(9, 0, 0),    # 09:00 UTC
    target_upper=time(17, 0, 0),   # 17:00 UTC
    use_task_logical_date=True,    # Use execution_date, not wall-clock
    dag=dag,
)

realtime_load = EmptyOperator(task_id="run_realtime_load")
batch_load    = EmptyOperator(task_id="run_batch_load")
converge      = EmptyOperator(
    task_id="converge",
```

```
    trigger_rule="none_failed_min_one_success",
)

branch_time >> [realtime_load, batch_load] >> converge
```

Calendar 1.10 BranchDayOfWeekOperator

```
from airflow.operators.weekday import BranchDayOfWeekOperator
```

> *Branches the pipeline based on the day of the week. Takes a set of WeekDay enum values and routes accordingly — no callable needed. Perfect for Monday-only full reloads, weekend skipping, or Friday-only reporting without custom Python.*

PYTHON

```python
from airflow.operators.weekday import BranchDayOfWeekOperator
from airflow.utils.weekday import WeekDay

# Run full reload only on Mondays
branch_day = BranchDayOfWeekOperator(
    task_id="check_day_of_week",
    follow_task_ids_if_true=["run_full_reload"],
    follow_task_ids_if_false=["run_incremental_load"],
    week_day={WeekDay.MONDAY},
    use_task_logical_date=True,
    dag=dag,
)

# Skip weekends entirely
branch_weekday = BranchDayOfWeekOperator(
    task_id="skip_weekends",
    follow_task_ids_if_true=["process_data"],
    follow_task_ids_if_false=["end"],
    week_day={
        WeekDay.MONDAY, WeekDay.TUESDAY, WeekDay.WEDNESDAY,
        WeekDay.THURSDAY, WeekDay.FRIDAY
    },
    dag=dag,
)
```

Catchup Guard 1.11 LatestOnlyOperator

```
from airflow.operators.latest_only import LatestOnlyOperator
```

> *A guard specifically for catchup and backfill scenarios. When multiple historical runs execute simultaneously, this operator skips all downstream tasks except for the most recently scheduled run . Use it to prevent old historical runs from triggering side effects — sending notifications, writing to production APIs, or refreshing dashboards multiple times. Place it between the data-loading tasks (which should backfill) and the notification/reporting tasks (which should only run once).*

PYTHON

```python
from airflow.operators.latest_only import LatestOnlyOperator
from airflow.operators.python import PythonOperator

with DAG(
    dag_id="catchup_safe_pipeline",
    schedule_interval="@daily",
```

```python
    start_date=datetime(2024, 1, 1),
    catchup=True,    # Historical backfill is intentional
) as dag:

    # Runs in ALL historical runs (data backfill is safe)
    load_historical = PythonOperator(
        task_id="load_to_data_lake",
        python_callable=load_raw_data,
    )

    # Gate: only the MOST RECENT run proceeds past this point
    latest_only = LatestOnlyOperator(task_id="latest_only_gate")

    # These run ONLY for the latest scheduled run
    send_report       = PythonOperator(task_id="send_daily_email_report",  python_callable=send_email)
    refresh_dashboard = PythonOperator(task_id="refresh_looker_dashboard", python_callable=trigger_looker_pdt)

    load_historical >> latest_only >> [send_report, refresh_dashboard]
```

Cross-DAG 1.12 TriggerDagRunOperator

```python
from airflow.operators.trigger_dagrun import TriggerDagRunOperator
```

> *Programmatically triggers another DAG run from within the current DAG. Supports passing a conf dict to the triggered DAG (readable as dag_run.conf in the child). The wait_for_completion=True flag makes this operator block until the child DAG finishes — enabling true parent-child pipeline dependencies. Combined with ExternalTaskSensor, this is the primary mechanism for cross-DAG orchestration.*

PYTHON

```python
from airflow.operators.trigger_dagrun import TriggerDagRunOperator

# ── Wait for child DAG to complete before moving on ──────────────────
trigger_reporting = TriggerDagRunOperator(
    task_id="trigger_reporting_dag",
    trigger_dag_id="generate_monthly_reports",
    conf={
        "run_date":    "{{ ds }}",
        "triggered_by": "{{ dag.dag_id }}",
        "environment":  "production",
    },
    wait_for_completion=True,
    poke_interval=30,
    allowed_states=["success"],
    failed_states=["failed"],
    reset_dag_run=True,
    dag=dag,
)

# ── Fire-and-forget (do not block) ────────────────────────────────────
trigger_cleanup = TriggerDagRunOperator(
    task_id="trigger_cleanup_job",
    trigger_dag_id="data_lake_cleanup",
    conf={"cutoff_date": "{{ macros.ds_add(ds, -90) }}"},
    wait_for_completion=False,
    dag=dag,
)

# ── Reading conf inside the TRIGGERED DAG ─────────────────────────────
def use_trigger_conf(**context):
```

```
    conf     = context["dag_run"].conf or {}
    run_date = conf.get("run_date", context["ds"])
    print(f"Triggered with run_date: {run_date}")
```

Structural 1.13 EmptyOperator

```
from airflow.operators.empty import EmptyOperator
```

A no-operation operator — does nothing on execution. Despite its simplicity, EmptyOperator is one of the most architecturally important tools in Airflow. It serves as: DAG entry/exit bookends, visual grouping boundaries in the graph view, convergence nodes after branching, synchronisation gates for parallel task groups, and placeholders for tasks not yet implemented.

PYTHON

```python
from airflow.operators.empty import EmptyOperator
from airflow.utils.trigger_rule import TriggerRule

# ── Structural bookends ──────────────────────────────────────────
start = EmptyOperator(task_id="start")
end   = EmptyOperator(task_id="end")

# ── Branch convergence node ──────────────────────────────────────
# NONE_FAILED_MIN_ONE_SUCCESS: succeeds as soon as ≥1 upstream succeeds
# and no upstream has failed. Handles skipped branches gracefully.
join = EmptyOperator(
    task_id="join_branches",
    trigger_rule=TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS,
)

# ── Fan-in synchroniser ──────────────────────────────────────────
# Wait for ALL parallel tasks to succeed before proceeding.
sync = EmptyOperator(
    task_id="sync_parallel_tasks",
    trigger_rule=TriggerRule.ALL_SUCCESS,   # default
)

# ── Placeholder (task not yet implemented) ───────────────────────
todo = EmptyOperator(task_id="future_ml_scoring")   # Replace later

# Typical DAG shape:
# start >> [task_a, task_b, task_c] >> sync >> process >> join >> end
```

Foundation Class 1.14 BaseOperator

```
from airflow.models.baseoperator import BaseOperator
```

Every operator in Airflow — from PythonOperator to BigQueryInsertJobOperator — inherits from BaseOperator . It is the abstract base class that defines the contract all operators must fulfil. The only method you must override is execute(context) . BaseOperator provides all the common operator infrastructure: retry logic, XCom push/pull, timeout enforcement, SLA tracking, templated field rendering, callback hooks (on_failure_callback , on_retry_callback , on_success_callback), dependency tracking, and resource pool integration. Understanding BaseOperator is essential for building custom operat…

PYTHON

```python
from airflow.models.baseoperator import BaseOperator
from airflow.utils.context import Context
```

```python
from typing import Any
import requests, json

# ── Every BaseOperator subclass must implement execute() ──────────────────
class SlackMessageOperator(BaseOperator):
    """
    Custom operator that posts a Slack message.
    Inherits all retry, callback, XCom, and templating from BaseOperator.
    """

    # Declare which attributes are Jinja-rendered at task runtime
    # {{ ds }}, {{ run_id }}, {{ params.env }}, etc. work in these fields
    template_fields = ("message", "channel")

    # Appear as a distinct color in the Airflow graph view
    ui_color  = "#2EA44F"    # GitHub green
    ui_fgcolor = "#FFFFFF"

    def __init__(
        self,
        message: str,
        channel: str = "#data-alerts",
        http_conn_id: str = "slack_webhook",
        **kwargs,                          # ALWAYS pass **kwargs to super().__init__()
    ) -> None:                             # This preserves retries, queue, pool etc.
        super().__init__(**kwargs)
        self.message      = message        # Jinja-rendered (in template_fields)
        self.channel      = channel        # Jinja-rendered (in template_fields)
        self.http_conn_id = http_conn_id

    def execute(self, context: Context) -> Any:
        """
        The single method you must override.
        context  → the full Airflow task context dict (ds, ti, dag_run, params…)
        Returns  → value is automatically pushed to XCom as return_value
        """
        from airflow.providers.http.hooks.http import HttpHook

        self.log.info("Posting to Slack channel: %s", self.channel)
        self.log.info("Message: %s", self.message)

        hook     = HttpHook(method="POST", http_conn_id=self.http_conn_id)
        payload  = json.dumps({"channel": self.channel, "text": self.message})
        response = hook.run(endpoint="", data=payload,
                            headers={"Content-Type": "application/json"})

        self.log.info("Slack response: %s", response.status_code)
        return response.status_code   # Pushed to XCom as return_value

    def on_kill(self) -> None:
        """Optional: called when the task is externally killed (e.g., DAG deleted mid-run)."""
        self.log.warning("SlackMessageOperator was killed — cleaning up")


# ── Using the custom operator ──────────────────────────────────────────────
notify = SlackMessageOperator(
    task_id="notify_pipeline_complete",
    message="✅ Pipeline *{{ dag.dag_id }}* completed for `{{ ds }}`",
    channel="#data-engineering",
    http_conn_id="slack_webhook",
    # BaseOperator kwargs — all work transparently:
    retries=2,
```

```python
    retry_delay=timedelta(seconds=30),
    execution_timeout=timedelta(minutes=2),
    on_failure_callback=slack_alert_on_failure,
    queue="default",
    dag=dag,
)

# ── Key BaseOperator parameters (shared by ALL operators) ──────────────────
# task_id             → unique identifier within the DAG (required)
# retries             → number of retry attempts after failure
# retry_delay         → wait time between retries (timedelta)
# execution_timeout   → max runtime before task is killed (timedelta)
# depends_on_past     → wait for same task in previous run to succeed
# trigger_rule        → when to run based on upstream states
# queue               → Celery queue name for worker routing
# pool                → resource pool for slot-based throttling
# pool_slots          → number of pool slots this task consumes (default 1)
# priority_weight     → scheduling priority when multiple tasks are queued
# weight_rule         → how to compute effective priority (downstream, upstream, absolute)
# on_failure_callback → callable called on task failure
# on_retry_callback   → callable called on task retry
# on_success_callback → callable called on task success
# on_skipped_callback → callable called when task is skipped (Airflow 2.7+)
# sla                 → timedelta — alert if task doesn't START within this window
# executor_config     → dict — per-task executor overrides (K8s resources, pod labels)
# do_xcom_push        → whether to push return value to XCom (default True)
# multiple_outputs    → if True, dict return value is pushed as separate XCom keys
```

Notifications 1.15 EmailOperator

```python
from airflow.operators.email import EmailOperator
```

> *Sends an email from within a DAG task using SMTP credentials stored in an Airflow Connection. Unlike email_on_failure=True in default_args (which auto-sends on failure), the EmailOperator is a first-class task that you explicitly wire into the dependency graph. This lets you send rich HTML emails at specific pipeline stages — upon successful completion, with computed metrics, with attached reports, or to different recipients based on branching logic. It is backed by SmtpHook internally.*

PYTHON

```python
from airflow.operators.email import EmailOperator
from airflow.decorators import dag, task
from datetime import datetime

@dag(schedule="@daily", start_date=datetime(2024, 1, 1), catchup=False)
def report_pipeline():

    @task
    def compute_kpis(ds: str) -> dict:
        return {
            "revenue":        1_284_500.75,
            "orders":         4_823,
            "avg_order_value":266.21,
            "date":           ds,
        }

    # ── Example 1: Plain HTML email with Jinja-templated content ──────────
    send_daily_report = EmailOperator(
        task_id="send_daily_kpi_report",
        to=["analytics@company.com", "ceo@company.com"],
```

```python
        cc=["data-engineering@company.com"],
        subject="[Daily KPI] Revenue Report — {{ ds }}",
        html_content="""
            Daily KPI Report — {{ ds }}

              MetricValue
              Revenue${{ ti.xcom_pull('compute_kpis')['revenue'] | round(2) }}
              Orders{{ ti.xcom_pull('compute_kpis')['orders'] }}
              Avg Order Value${{ ti.xcom_pull('compute_kpis')['avg_order_value'] }}

            Generated by Airflow DAG: {{ dag.dag_id }} | Run: {{ run_id }}
        """,
        conn_id="smtp_prod",   # SMTP connection stored in Airflow
        mime_charset="utf-8",
    )

    # ── Example 2: Email with file attachment ──────────────────────────────
    @task
    def generate_csv_report(kpis: dict) -> str:
        import csv, io, tempfile
        tmp = tempfile.NamedTemporaryFile(suffix=".csv", delete=False, mode="w")
        writer = csv.DictWriter(tmp, fieldnames=kpis.keys())
        writer.writeheader()
        writer.writerow(kpis)
        tmp.close()
        return tmp.name

    send_with_attachment = EmailOperator(
        task_id="send_report_with_csv",
        to=["finance@company.com"],
        subject="[Finance] Daily Orders CSV — {{ ds }}",
        html_content="Please find today's order summary attached.",
        files=["{{ ti.xcom_pull('generate_csv_report') }}"],   # Jinja path to file
        conn_id="smtp_prod",
    )

    kpis   = compute_kpis()
    csv_path = generate_csv_report(kpis)
    kpis >> send_daily_report
    csv_path >> send_with_attachment

dag_instance = report_pipeline()
```

Extend Airflow 1.16 Custom Operators

```python
from airflow.models.baseoperator import BaseOperator
```

> *Custom Operators allow you to package reusable task logic into a single, testable, discoverable class. Instead of repeating the same 30-line PythonOperator callable in dozens of DAGs, you build it once as a class, register it via the plugin system or as a Python package, and use it across all DAGs with a clean, documented API. Custom operators also integrate naturally with Jinja templating via template_fields , appear with a custom color in the UI, and work transparently with all BaseOperator features (retry, XCom, callbacks, pools).*

PYTHON

```python
# plugins/operators/dbt_cloud_operator.py
from __future__ import annotations
from airflow.models.baseoperator import BaseOperator
from airflow.utils.context import Context
from typing import Any
```

```python
import time

class DbtCloudRunJobOperator(BaseOperator):
    """
    Triggers a dbt Cloud job run and (optionally) waits for it to complete.

    Connection 'dbt_cloud' must be set up with:
      - host: https://cloud.getdbt.com
      - extra: {"Authorization": "Token your-service-token"}

    :param account_id: dbt Cloud account ID
    :param job_id:     dbt Cloud job ID to trigger
    :param cause:      Human-readable reason for the run (Jinja-rendered)
    :param wait:       If True, poll until the job completes
    :param timeout_s:  Max seconds to wait before raising TimeoutError
    """

    # These fields are Jinja-rendered at runtime — {{ ds }}, {{ run_id }}, etc. work
    template_fields = ("cause", "additional_run_config")

    ui_color   = "#FF694B"   # dbt orange
    ui_fgcolor = "#FFFFFF"

    def __init__(
        self,
        account_id: int,
        job_id:     int,
        cause:      str  = "Triggered by Airflow run {{ run_id }}",
        wait:       bool = True,
        timeout_s:  int  = 3600,
        poll_interval: int = 30,
        dbt_conn_id: str = "dbt_cloud",
        additional_run_config: dict | None = None,
        **kwargs,
    ) -> None:
        super().__init__(**kwargs)   # ← ALWAYS pass **kwargs
        self.account_id           = account_id
        self.job_id               = job_id
        self.cause                = cause
        self.wait                 = wait
        self.timeout_s            = timeout_s
        self.poll_interval        = poll_interval
        self.dbt_conn_id          = dbt_conn_id
        self.additional_run_config = additional_run_config or {}

    def execute(self, context: Context) -> dict:
        from airflow.providers.http.hooks.http import HttpHook
        import json

        hook = HttpHook(method="POST", http_conn_id=self.dbt_conn_id)

        # Trigger the job
        payload = {
            "cause": self.cause,
            **self.additional_run_config,
        }
        response = hook.run(
            endpoint=f"/api/v2/accounts/{self.account_id}/jobs/{self.job_id}/run/",
            data=json.dumps(payload),
            headers={"Content-Type": "application/json"},
        )
        run_data = response.json()["data"]
```

```python
        run_id   = run_data["id"]
        self.log.info("dbt Cloud job %d started — run ID: %d", self.job_id, run_id)

        if not self.wait:
            return {"run_id": run_id, "status": "triggered"}

        # Poll until complete
        get_hook  = HttpHook(method="GET", http_conn_id=self.dbt_conn_id)
        start     = time.monotonic()
        while True:
            if time.monotonic() - start > self.timeout_s:
                raise TimeoutError(f"dbt Cloud run {run_id} did not complete in {self.timeout_s}s")

            status_resp = get_hook.run(
                endpoint=f"/api/v2/accounts/{self.account_id}/runs/{run_id}/",
            )
            run_status = status_resp.json()["data"]["status_humanized"]
            self.log.info("dbt Cloud run %d status: %s", run_id, run_status)

            if run_status == "Success":
                self.log.info("dbt Cloud run %d completed successfully", run_id)
                return {"run_id": run_id, "status": "success"}
            if run_status in ("Error", "Cancelled"):
                raise Exception(f"dbt Cloud run {run_id} failed: {run_status}")

            time.sleep(self.poll_interval)

# ── Register via plugin ──────────────────────────────────────────────
from airflow.plugins_manager import AirflowPlugin

class DbtPlugin(AirflowPlugin):
    name      = "dbt_plugin"
    operators = [DbtCloudRunJobOperator]

# ── Usage in a DAG ───────────────────────────────────────────────────
# from plugins.operators.dbt_cloud_operator import DbtCloudRunJobOperator
#
# run_dbt = DbtCloudRunJobOperator(
#     task_id="run_dbt_daily_models",
#     account_id=12345,
#     job_id=678,
#     cause="Triggered by Airflow DAG {{ dag.dag_id }} for {{ ds }}",
#     wait=True,
#     timeout_s=1800,
#     retries=1,
#     retry_delay=timedelta(minutes=2),
# )
```

PYTHON

```python
# tests/test_dbt_cloud_operator.py
import pytest
from unittest.mock import MagicMock, patch
from plugins.operators.dbt_cloud_operator import DbtCloudRunJobOperator

class TestDbtCloudRunJobOperator:

    def test_template_fields_declared(self):
        """Ensure Jinja template_fields are declared."""
        assert "cause" in DbtCloudRunJobOperator.template_fields

    @patch("airflow.providers.http.hooks.http.HttpHook.run")
```

```python
    def test_execute_triggers_and_returns_run_id(self, mock_run):
        """Mock the HTTP calls and verify the operator returns the run ID."""
        # Setup mock responses
        trigger_resp = MagicMock()
        trigger_resp.json.return_value = {"data": {"id": 999}}
        status_resp  = MagicMock()
        status_resp.json.return_value  = {"data": {"status_humanized": "Success"}}
        mock_run.side_effect = [trigger_resp, status_resp]

        op      = DbtCloudRunJobOperator(
            task_id="test_dbt",
            account_id=123,
            job_id=456,
            wait=True,
            poll_interval=0,   # No sleep in tests
        )
        context = {"run_id": "test_run_001", "ds": "2025-01-15"}
        result  = op.execute(context)

        assert result["run_id"] == 999
        assert result["status"] == "success"
        assert mock_run.call_count == 2   # trigger + one status poll

    def test_raises_on_timeout(self):
        """Operator should raise TimeoutError when run doesn't complete in time."""
        with patch("airflow.providers.http.hooks.http.HttpHook.run") as mock_run:
            trigger_resp = MagicMock()
            trigger_resp.json.return_value = {"data": {"id": 999}}
            running_resp = MagicMock()
            running_resp.json.return_value = {"data": {"status_humanized": "Running"}}
            mock_run.side_effect = [trigger_resp] + [running_resp] * 100

            op = DbtCloudRunJobOperator(
                task_id="test_dbt", account_id=123, job_id=456,
                wait=True, timeout_s=0, poll_interval=0,
            )
            with pytest.raises(TimeoutError):
                op.execute({"run_id": "test", "ds": "2025-01-15"})
```

Efficient Waiting 1.17 Deferrable Operators & Async Execution

```python
from airflow.sensors.base import BaseSensorOperator | from airflow.triggers.base import BaseTrigger
```

> *Deferrable Operators (introduced in Airflow 2.2) allow a task to suspend itself , release its worker slot, and wait for an async condition to be met — all without occupying a Celery or Kubernetes worker process. The Triggerer component runs thousands of async I/O loops simultaneously (using Python asyncio) to watch for conditions. When the condition fires, the Triggerer re-queues the task. This dramatically reduces worker slot consumption for long-waiting tasks and is the production-correct way to implement sensors with wait times exceeding a few minutes.*

PYTHON

```python
from airflow.decorators import dag, task
from datetime import datetime, timedelta

# ── Using built-in deferrable sensor variants ──────────────────────────
# Most provider sensors now have deferrable=True parameter or a separate Deferrable class.

from airflow.providers.google.cloud.sensors.gcs import GCSObjectExistenceSensor

@dag(schedule="@daily", start_date=datetime(2024, 1, 1), catchup=False)
```

```python
def deferrable_demo():

    # ── Standard sensor (mode=reschedule) — still uses a worker slot between polls
    standard_sensor = GCSObjectExistenceSensor(
        task_id="wait_file_standard",
        bucket="data-lake",
        object="raw/{{ ds }}/data.parquet",
        mode="reschedule",      # Releases worker between polls — better than poke
        poke_interval=120,
        timeout=28800,
    )

    # ── Deferrable sensor — releases worker slot ENTIRELY until file appears
    # The Triggerer monitors for the file using async I/O — zero worker slot usage
    deferrable_sensor = GCSObjectExistenceSensor(
        task_id="wait_file_deferrable",
        bucket="data-lake",
        object="raw/{{ ds }}/data.parquet",
        deferrable=True,        # Use async Triggerer — no worker slot held
        poke_interval=120,      # How often the async trigger checks
        timeout=28800,
    )

    deferrable_sensor   # This is the correct production choice for long waits

dag_instance = deferrable_demo()


# ── Building a custom Deferrable Operator ──────────────────────────────────
from airflow.models.baseoperator import BaseOperator
from airflow.triggers.base import BaseTrigger, TriggerEvent
from airflow.utils.context import Context
import asyncio, aiohttp

class HttpJobTrigger(BaseTrigger):
    """
    Async trigger that polls an HTTP endpoint until the job completes.
    Runs inside the Triggerer process — must use async/await.
    """

    def __init__(self, job_id: str, conn_id: str, poll_interval: int = 30):
        super().__init__()
        self.job_id        = job_id
        self.conn_id       = conn_id
        self.poll_interval = poll_interval

    def serialize(self) -> tuple:
        """Required: return (class_path, kwargs) for serialisation."""
        return (
            "plugins.triggers.HttpJobTrigger",
            {"job_id": self.job_id, "conn_id": self.conn_id,
             "poll_interval": self.poll_interval},
        )

    async def run(self):
        """
        The async loop — runs in the Triggerer (not a worker).
        yield TriggerEvent when done.
        """
        while True:
            async with aiohttp.ClientSession() as session:
                async with session.get(
```

```
                    f"https://api.example.com/jobs/{self.job_id}/status"
                ) as resp:
                    data = await resp.json()
                    status = data.get("status")

            if status == "completed":
                yield TriggerEvent({"status": "success", "job_id": self.job_id})
                return
            if status in ("failed", "cancelled"):
                yield TriggerEvent({"status": "failed",  "job_id": self.job_id})
                return

            await asyncio.sleep(self.poll_interval)


class DeferringHttpJobOperator(BaseOperator):
    """
    Submits an HTTP job and then defers — releases the worker slot
    while the Triggerer monitors for completion.
    """

    def execute(self, context: Context):
        """Submit the job, then defer immediately."""
        import requests
        resp   = requests.post("https://api.example.com/jobs",
                                json={"type": "etl", "date": context["ds"]})
        job_id = resp.json()["job_id"]
        self.log.info("Job submitted: %s — deferring now", job_id)

        # self.defer() suspends this task and releases the worker slot.
        # The Triggerer runs HttpJobTrigger.run() asynchronously.
        # When TriggerEvent fires, execute_complete() is called.
        self.defer(
            trigger=HttpJobTrigger(job_id=job_id, conn_id="my_api"),
            method_name="execute_complete",      # Called when trigger fires
            timeout=timedelta(hours=4),          # Fail if trigger never fires
        )

    def execute_complete(self, context: Context, event: dict) -> dict:
        """
        Called by the Triggerer when the async condition is met.
        event  → the TriggerEvent payload from HttpJobTrigger.run()
        """
        if event["status"] == "failed":
            raise Exception(f"Job {event['job_id']} failed")
        self.log.info("Job %s completed successfully", event["job_id"])
        return event   # Pushed to XCom
```

Airflow 2.3+ 1.18 MappedOperator — Dynamic Task Mapping

```
from airflow.decorators import task | operator.expand () / expand_kwargs ()
```

> *Dynamic Task Mapping (Airflow 2.3+) generates task instances at runtime based on the output of an upstream task — you don't know how many tasks will run until the DAG is executing. This replaces the pre-Airflow 2.3 pattern of using Python loops to create tasks at parse time (which required knowing the number of tasks when writing the DAG file). The resulting operator instances are called MappedOperators . Each mapped instance runs independently, in parallel, in its own worker slot, and reports its own state in the UI.*

PARAMETER	TYPE / DEFAULT	DESCRIPTION
Parallelism	All mapped instances run concurrently (up to max_active_tasks limit)	
Independent state	Each instance has its own state — one failure doesn't block others (default)	
Retries	Each instance retries independently with its own try_number	
Max map size	Configurable via max_map_length in airflow.cfg (default: 1024)	
UI visibility	Grid view shows each instance as a separate row with expand/collapse	
Downstream aggregation	A downstream task receives a list of all mapped outputs via XCom	
expand_kwargs() limit	Each dict key must match a parameter of the callable exactly	
trigger_rule on downstream	Use ALL_SUCCESS (default) or ALL_DONE to handle partial failures	
Accessing map_index	context["ti"].map_index gives the 0-based index of the current instance	
Zero-length mapping	If expand input is empty list [], no tasks are created and downstream is skipped	

PYTHON

```python
from airflow.decorators import dag, task
from datetime import datetime

@dag(schedule="@daily", start_date=datetime(2024, 1, 1), catchup=False,
     tags=["dynamic-mapping"])
def dynamic_task_mapping_reference():

    # ── Pattern 1: expand() — map over a single argument list ──────────────
    @task
    def list_gcs_files(ds: str) -> list[str]:
        """Returns a list at runtime — one task will be created per element."""
        from airflow.providers.google.cloud.hooks.gcs import GCSHook
        hook  = GCSHook(gcp_conn_id="google_cloud_default")
        files = hook.list(bucket_name="data-lake", prefix=f"raw/{ds}/")
        print(f"Found {len(files)} files for {ds}")
        return files   # e.g., ["raw/2025-01-15/part-001.parquet", ...]

    @task
    def process_file(file_path: str) -> dict:
        """
        This is instantiated ONCE PER FILE at runtime.
        Each instance runs independently, in parallel, with its own retry state.
        """
        print(f"Processing: {file_path}")
        # ... actual file processing ...
        return {"file": file_path, "rows": 1000, "status": "done"}

    @task
    def aggregate_results(results: list[dict]) -> dict:
        """
        Receives a LIST of all mapped task outputs.
        Runs after ALL mapped instances complete (or based on trigger_rule).
        """
        total_rows = sum(r["rows"] for r in results)
        failed     = [r["file"] for r in results if r["status"] != "done"]
        print(f"Total: {total_rows} rows across {len(results)} files. Failures: {failed}")
        return {"total_rows": total_rows, "file_count": len(results)}

    files   = list_gcs_files()
    results = process_file.expand(file_path=files)   # .expand() creates N mapped tasks
    aggregate_results(results)
```

```python
    # ── Pattern 2: expand_kwargs() — map over a list of dicts ──────────────
    @task
    def get_tenant_configs() -> list[dict]:
        return [
            {"tenant_id": "acme",    "bq_dataset": "acme_prod",    "region": "US"},
            {"tenant_id": "globex",  "bq_dataset": "globex_prod",  "region": "EU"},
            {"tenant_id": "initech", "bq_dataset": "initech_prod", "region": "US"},
        ]

    @task
    def sync_tenant(tenant_id: str, bq_dataset: str, region: str) -> str:
        print(f"[{tenant_id}] Syncing {bq_dataset} in {region}")
        return f"{tenant_id}:ok"

    configs  = get_tenant_configs()
    statuses = sync_tenant.expand_kwargs(configs)  # One task per dict in the list


    # ── Pattern 3: partial() + expand() — fix some args, map others ──────────
    @task
    def load_to_bq(table: str, project: str, dataset: str) -> dict:
        print(f"Loading to {project}.{dataset}.{table}")
        return {"table": table, "status": "loaded"}

    tables = ["orders", "customers", "products", "inventory"]

    # project and dataset are FIXED; table is MAPPED (one task per table)
    load_results = load_to_bq.partial(
        project="my-project",
        dataset="prod_raw",
    ).expand(table=tables)   # Creates 4 tasks: load_orders, load_customers, etc.


    # ── Pattern 4: map_index_template — custom mapped task IDs ──────────────
    @task(map_index_template="{{ task.op_kwargs['table'] }}")
    def transform_table(table: str) -> None:
        """
        By default mapped tasks appear as transform_table[0], transform_table[1]...
        map_index_template renames them to transform_table[orders], etc.
        """
        print(f"Transforming: {table}")

    transform_table.expand(table=["orders", "customers", "products"])

dag_instance = dynamic_task_mapping_reference()
```

CH 2 Sensors — Complete Reference

Sensors are specialised operators that pause the pipeline until a condition is met. They run in poke, reschedule, or deferrable mode. Always use reschedule or deferrable in production to avoid occupying worker slots during long waits.

9 sensors covered in this chapter

Filesystem 2.1 FileSensor / FileSystemSensor

```
from airflow.sensors.filesystem import FileSensor
```

Polls the local worker filesystem for the existence of a file or directory. The path is Jinja-rendered, allowing dynamic date-based paths. FileSystemSensor is a legacy alias — both are identical. For cloud storage, use provider-specific sensors (GCSObjectExistenceSensor, S3KeySensor) instead.

PYTHON

```python
from airflow.sensors.filesystem import FileSensor

# Wait for a specific file with exponential backoff
wait_for_csv = FileSensor(
    task_id="wait_for_daily_export",
    filepath="/data/exports/{{ ds }}/transactions.csv",
    fs_conn_id="fs_default",
    mode="reschedule",
    poke_interval=120,
    timeout=28800,
    exponential_backoff=True,   # 120s, 240s, 480s… between checks
    dag=dag,
)
```

Cross-DAG 2.2 ExternalTaskSensor

```
from airflow.sensors.external_task import ExternalTaskSensor
```

Blocks until a task (or entire DAG run) in a different DAG reaches a target state. Matches by execution_date by default. Use execution_delta or execution_date_fn when the two DAGs run on different schedules. This is the foundation of cross-DAG dependency management.

PYTHON

```python
from airflow.sensors.external_task import ExternalTaskSensor
from datetime import timedelta

# Wait for a specific task in another DAG
wait_ingestion = ExternalTaskSensor(
    task_id="wait_for_ingestion",
    external_dag_id="data_ingestion_pipeline",
    external_task_id="validate_schema",
    allowed_states=["success"],
    failed_states=["failed", "skipped"],
    execution_delta=timedelta(hours=1),
    mode="reschedule",
    poke_interval=60,
    timeout=7200,
    dag=dag,
)
```

```python
# Wait for the ENTIRE upstream DAG (external_task_id=None)
wait_full_dag = ExternalTaskSensor(
    task_id="wait_for_upstream_dag",
    external_dag_id="upstream_etl_dag",
    external_task_id=None,   # Wait for entire DAG run
    mode="reschedule",
    timeout=14400,
    dag=dag,
)

# Cross-schedule: upstream hourly, this DAG daily
def map_to_last_hourly(execution_date, **kwargs):
    return execution_date.replace(hour=23, minute=0, second=0)

wait_last_hourly = ExternalTaskSensor(
    task_id="wait_for_last_hourly",
    external_dag_id="hourly_events_pipeline",
    external_task_id="load_events",
    execution_date_fn=map_to_last_hourly,
    mode="reschedule",
    timeout=3600,
    dag=dag,
)
```

Relative Time 2.3 TimeDeltaSensor

```python
from airflow.sensors.time_delta import TimeDeltaSensor
```

> *Waits for a specified timedelta to elapse from the execution_date . Unlike TimeSensor (absolute), TimeDeltaSensor waits a RELATIVE duration. Use it to implement minimum lag requirements — ensuring data is at least 2 hours old before processing.*

PYTHON

```python
from airflow.sensors.time_delta import TimeDeltaSensor
from datetime import timedelta

# Wait 2 hours after execution_date (let source system finish writing)
wait_lag = TimeDeltaSensor(
    task_id="wait_2hr_lag",
    delta=timedelta(hours=2),
    mode="reschedule",
    poke_interval=120,
    dag=dag,
)
```

Absolute Time 2.4 TimeSensor

```python
from airflow.sensors.time import TimeSensor
```

> *Blocks execution until the wall-clock time (UTC) reaches the specified target_time — an absolute time of day. Unlike TimeDeltaSensor (relative offset), TimeSensor waits for a specific daily clock time. Useful for maintenance windows or SLA-driven deadlines.*

PYTHON

```python
from airflow.sensors.time import TimeSensor
from datetime import time
```

```python
# Block until 08:00 UTC — wait for overnight batch to complete
wait_for_morning = TimeSensor(
    task_id="wait_until_08_utc",
    target_time=time(8, 0, 0),
    mode="reschedule",
    poke_interval=300,
    dag=dag,
)
```

Datetime Gate 2.5 DateTimeSensor

```python
from airflow.sensors.date_time import DateTimeSensor
```

Waits until a specific absolute datetime (date + time) is reached. More powerful than TimeSensor — the target_time can be a Jinja-rendered string, enabling dynamic deadlines based on the execution_date. Use for hard release gates or time-bounded maintenance schedules.

PYTHON

```python
from airflow.sensors.date_time import DateTimeSensor
import pendulum

# Wait until end-of-day on the execution date
wait_eod = DateTimeSensor(
    task_id="wait_for_eod_dynamic",
    target_time="{{ execution_date.replace(hour=23, minute=55, second=0) }}",
    mode="reschedule",
    poke_interval=300,
    dag=dag,
)
```

Shell Poll 2.6 BashSensor

```python
from airflow.sensors.bash import BashSensor
```

Repeatedly executes a Bash command and succeeds (stops waiting) when it returns exit code 0. Trivially adapts any command-line check into a sensor — ping an endpoint, check an FTP listing, poll a lock file — without writing Python.

PYTHON

```python
from airflow.sensors.bash import BashSensor

# Poll until a GCS done-flag file appears via gsutil
wait_gcs_flag = BashSensor(
    task_id="wait_for_gcs_flag",
    bash_command="gsutil stat gs://data-bucket/exports/{{ ds }}/done.flag",
    mode="reschedule",
    poke_interval=120,
    timeout=14400,
    dag=dag,
)
```

Custom Logic 2.7 PythonSensor

```python
from airflow.sensors.python import PythonSensor
```

> *The most flexible sensor type. Repeatedly calls a Python callable and succeeds when it returns True . The callable receives the full Airflow context and can use any hook, library, or API to check an arbitrary condition. The truthy return value is also pushed to XCom, letting sensed data flow downstream.*

PYTHON

```python
from airflow.sensors.python import PythonSensor

def check_api_data_ready(ds: str, **context) -> bool:
    """Return True to stop sensing, False to keep waiting."""
    import requests
    response = requests.get(
        "https://api.partner.com/v2/data-status",
        params={"date": ds, "source": "orders"},
        headers={"Authorization": "Bearer {{ var.value.partner_api_key }}"},
        timeout=10,
    )
    if response.status_code == 200:
        return response.json().get("status") == "complete"
    return False

wait_api = PythonSensor(
    task_id="wait_for_partner_api",
    python_callable=check_api_data_ready,
    provide_context=True,
    mode="reschedule",
    poke_interval=180,
    timeout=14400,
    soft_fail=True,   # SKIP (not FAIL) if timeout exceeded
    dag=dag,
)
```

SQL Gate 2.8 SqlSensor

```python
from airflow.providers.common.sql.sensors.sql import SqlSensor
```

> *Executes a SQL query and waits until it returns a non-null, non-zero, non-false value. Use as a data availability gate — wait until a staging table has records, a partition exists, or an aggregation exceeds a threshold. Supports PostgreSQL, MySQL, Redshift, Snowflake, BigQuery, and any SQLAlchemy-compatible database.*

PYTHON

```python
from airflow.providers.common.sql.sensors.sql import SqlSensor

# Wait until source table has data for today
wait_source = SqlSensor(
    task_id="wait_for_source_data",
    conn_id="postgres_prod",
    sql="""
        SELECT COUNT(*) FROM raw.transactions
        WHERE transaction_date = '{{ ds }}'
          AND status = 'committed'
    """,
    mode="reschedule",
    poke_interval=120,
    timeout=14400,
```

```python
    dag=dag,
)

# Wait for BigQuery partition with custom success condition
wait_bq = SqlSensor(
    task_id="wait_for_bq_partition",
    conn_id="google_cloud_default",
    sql="SELECT COUNT(*) FROM `project.dataset.fact_orders` WHERE DATE(_PARTITIONTIME)='{{ ds }}'",
    success=lambda row: row[0][0] > 1000,   # Custom condition
    mode="reschedule",
    poke_interval=300,
    dag=dag,
)
```

HTTP Poll 2.9 HttpSensor

```python
from airflow.providers.http.sensors.http import HttpSensor
```

Polls an HTTP endpoint and succeeds when the response meets a condition. By default, any 2xx response triggers success. A response_check callable inspects the response body for a specific JSON field or status string. Backed by HttpHook, using the http_conn_id Connection for the base URL and auth headers.

PYTHON

```python
from airflow.providers.http.sensors.http import HttpSensor

# Poll until API job reaches "completed" status
wait_job = HttpSensor(
    task_id="wait_for_api_job",
    http_conn_id="partner_api",
    endpoint="v1/jobs/{{ ti.xcom_pull(task_ids='submit_job') }}/status",
    method="GET",
    response_check=lambda r: r.json().get("status") == "completed",
    extra_options={"timeout": 10},
    mode="reschedule",
    poke_interval=60,
    timeout=7200,
    dag=dag,
)
```

CHAPTER 3 OF 3

CH 3 Hooks — Complete Reference

Hooks are the credential-aware connection layer. They read from Airflow's Connection store, handle auth, and expose high-level methods for external systems. Operators use Hooks internally — and you can call them directly in @task callables for full control.

10 hooks covered in this chapter

Abstract Base 3.1 BaseHook

```
from airflow.hooks.base import BaseHook
```

The abstract base class for every hook in Airflow. Provides get_connection(conn_id) to read from the metadata database or secrets backend, and test_connection() for UI connectivity testing. Extend this when building custom hooks — override get_conn() to return a ready-to-use client object.

PYTHON

```python
from airflow.hooks.base import BaseHook

# Read a connection directly
conn = BaseHook.get_connection("postgres_prod")
print(conn.host, conn.port, conn.login, conn.schema)

# Build a custom Hook
class MyApiHook(BaseHook):
    conn_name_attr   = "my_api_conn_id"
    default_conn_name = "my_api_default"
    conn_type        = "http"
    hook_name        = "My Partner API"

    def __init__(self, my_api_conn_id: str = default_conn_name) -> None:
        super().__init__()
        self.my_api_conn_id = my_api_conn_id

    def get_conn(self):
        import requests
        conn    = self.get_connection(self.my_api_conn_id)
        session = requests.Session()
        session.headers.update({"Authorization": f"Bearer {conn.password}"})
        return session

    def get_records(self, endpoint: str) -> list:
        session  = self.get_conn()
        response = session.get(f"https://{self.get_connection(self.my_api_conn_id).host}/{endpoint}")
        response.raise_for_status()
        return response.json().get("data", [])

    def test_connection(self):
        try:
            self.get_records("health")
            return True, "Connection successful"
        except Exception as e:
            return False, str(e)
```

Filesystem 3.2 FSHook

```python
from airflow.hooks.filesystem import FSHook
```

> *Reads a filesystem path prefix from an Airflow Connection of type "fs". Primarily used internally by FileSensor. Useful when you want to abstract away mount points and make them configurable per environment without modifying DAG code.*

```python
from airflow.hooks.filesystem import FSHook

def check_and_read_file(ds: str, **context) -> dict:
    hook      = FSHook(fs_conn_id="fs_default")
    base      = hook.get_path()     # "/data/exports" from Connection extra JSON
    full_path = f"{base}/{ds}/transactions.csv"

    import os, csv
    if not os.path.exists(full_path):
        raise FileNotFoundError(f"Not found: {full_path}")

    with open(full_path) as f:
        rows = list(csv.DictReader(f))

    return {"path": full_path, "rows": len(rows)}
```

Shell 3.3 SubprocessHook

```python
from airflow.hooks.subprocess import SubprocessHook
```

> *Low-level hook for running shell subprocesses from within Python callables. Used internally by BashOperator. Exposes run_command() which streams stdout/stderr to task logs in real-time and returns a SubprocessResult with exit code and output. Use when building custom operators that need shell commands with full log streaming.*

```python
from airflow.hooks.subprocess import SubprocessHook

def run_dbt_command(command: str, **context) -> str:
    hook   = SubprocessHook()
    result = hook.run_command(
        command=["bash", "-c", command],
        env={
            "DBT_PROFILES_DIR": "/opt/dbt/profiles",
            "RUN_DATE": context["ds"],
        },
        cwd="/opt/dbt",
    )
    if result.exit_code != 0:
        raise RuntimeError(f"dbt failed (exit {result.exit_code}): {result.output}")
    return result.output
```

Kubernetes 3.4 KubernetesHook

```python
from airflow.providers.cncf.kubernetes.hooks.kubernetes import KubernetesHook
```

Provides a client for the Kubernetes API server. Used internally by KubernetesPodOperator. Directly usable in Python tasks when you need to query pod status, list jobs, read ConfigMaps, or manage K8s resources beyond what built-in operators expose.

```python
from airflow.providers.cncf.kubernetes.hooks.kubernetes import KubernetesHook
from kubernetes import client as k8s

def scale_deployment(name: str, replicas: int) -> None:
    hook   = KubernetesHook(conn_id="kubernetes_default")
    client = hook.get_conn()
    apps   = k8s.AppsV1Api(api_client=client)
    apps.patch_namespaced_deployment_scale(
        name=name,
        namespace="data-platform",
        body={"spec": {"replicas": replicas}},
    )
    print(f"Scaled {name} to {replicas} replicas")
```

Docker 3.5 DockerHook

```python
from airflow.providers.docker.hooks.docker import DockerHook
```

Wraps the Docker SDK and authenticates with a Docker registry using credentials from an Airflow Connection. Returns a docker.APIClient . Used internally by DockerOperator. Use directly when you need to pull images, inspect containers, manage volumes, or build custom Docker-based workflows.

```python
from airflow.providers.docker.hooks.docker import DockerHook
import docker

def run_container(image: str, command: str, **context) -> str:
    client = docker.DockerClient.from_env()
    output = client.containers.run(
        image=image,
        command=command,
        remove=True,
        environment={"RUN_DATE": context["ds"], "ENV": "production"},
        volumes={"/data": {"bind": "/data", "mode": "ro"}},
    )
    return output.decode("utf-8").strip()
```

REST / HTTP 3.6 HttpHook

```python
from airflow.providers.http.hooks.http import HttpHook
```

Provides a reusable HTTP/HTTPS client backed by an Airflow Connection that stores the base URL, authentication headers, and optional SSL settings. Wraps the requests library and supports GET, POST, PUT, PATCH, DELETE with automatic retry logic and response validation. Use this hook whenever your task needs to call a REST API, trigger a webhook, or interact with any HTTP endpoint — without hardcoding the base URL or credentials in your DAG file.

```python
from airflow.providers.http.hooks.http import HttpHook
```

```python
# ── Scenario 1: Trigger a dbt Cloud job via REST API ──────────────────────
def trigger_dbt_cloud_job(job_id: int, **context) -> dict:
    """
    Uses HttpHook to trigger a dbt Cloud job and return the run ID.
    Connection 'dbt_cloud_api' stores:
      host: https://cloud.getdbt.com
      extra: {"Authorization": "Token your-dbt-service-token"}
    """
    hook = HttpHook(method="POST", http_conn_id="dbt_cloud_api")
    response = hook.run(
        endpoint=f"/api/v2/accounts/12345/jobs/{job_id}/run/",
        data={"cause": f"Triggered by Airflow — run {context['run_id']}"},
        headers={"Content-Type": "application/json"},
        extra_options={"timeout": 30},
    )
    run_id = response.json()["data"]["id"]
    print(f"dbt Cloud job {job_id} started → run ID: {run_id}")
    return {"dbt_run_id": run_id, "status": "started"}

# ── Scenario 2: Poll an external API until status = complete ──────────────
def poll_dbt_run_until_done(run_id: int, **context) -> str:
    """
    Use HttpHook for custom polling logic with exponential backoff.
    Sensor operators do the same but this gives full control.
    """
    import time
    hook  = HttpHook(method="GET", http_conn_id="dbt_cloud_api")
    delay = 30

    for attempt in range(20):   # max 20 polls
        response = hook.run(
            endpoint=f"/api/v2/accounts/12345/runs/{run_id}/",
            headers={"Content-Type": "application/json"},
        )
        status = response.json()["data"]["status_humanized"]
        print(f"[Attempt {attempt+1}] dbt run {run_id} status: {status}")

        if status == "Success":
            return "success"
        if status in ("Error", "Cancelled"):
            raise Exception(f"dbt run {run_id} failed with status: {status}")

        time.sleep(delay)
        delay = min(delay * 1.5, 300)   # cap at 5 min

    raise TimeoutError(f"dbt run {run_id} did not complete in time")

# ── Scenario 3: POST data to a webhook ────────────────────────────────────
def send_webhook_notification(payload: dict, **context) -> None:
    """
    Notify an external monitoring system after pipeline completes.
    Connection 'monitoring_webhook' stores the base URL + auth token.
    """
    hook     = HttpHook(method="POST", http_conn_id="monitoring_webhook")
    response = hook.run(
        endpoint="/v1/events/pipeline-complete",
        data={
            "pipeline":   context["dag"].dag_id,
            "run_id":     context["run_id"],
            "date":       context["ds"],
            "status":     "success",
            "extra":      payload,
```

```
        },
        headers={"Content-Type": "application/json"},
        extra_options={"timeout": 10, "verify": True},
    )
    print(f"Webhook notified: HTTP {response.status_code}")
```

FTP / FTPS 3.7 FTPHook

```
from airflow.providers.ftp.hooks.ftp import FTPHook
```

> *Wraps Python's ftplib.FTP and provides a clean interface for interacting with FTP and FTPS servers using credentials stored in an Airflow Connection. Exposes methods for listing directories, downloading files, uploading files, creating directories, and deleting remote files. Connection type is ftp and stores host, port, login, and password. Use FTPSHook (same module) for encrypted FTPS connections.*

PYTHON

```python
from airflow.providers.ftp.hooks.ftp import FTPHook
from airflow.providers.google.cloud.hooks.gcs import GCSHook
import io, os

# ── Scenario 1: Download files from partner FTP and stage to GCS ──────────────
def ingest_from_partner_ftp(ds: str, **context) -> list:
    """
    Check partner FTP for new files matching today's date pattern,
    download them, and upload directly to GCS — no local disk needed.

    Connection 'partner_ftp' stores:
      host: ftp.partner.com, login: user, password: pass, port: 21
    """
    ftp_hook = FTPHook(ftp_conn_id="partner_ftp")
    gcs_hook = GCSHook(gcp_conn_id="google_cloud_default")

    ds_nodash = ds.replace("-", "")
    remote_dir = f"/outbound/orders/{ds_nodash}/"
    uploaded   = []

    with ftp_hook.get_conn() as ftp:
        ftp.cwd(remote_dir)
        files = ftp.nlst()   # list all filenames in the directory
        print(f"Found {len(files)} files in {remote_dir}")

        for filename in files:
            if not filename.endswith(".csv"):
                continue

            # Stream file content into memory (no temp file needed)
            buf = io.BytesIO()
            ftp.retrbinary(f"RETR {filename}", buf.write)
            buf.seek(0)

            gcs_path = f"raw/partner-ftp/{ds}/{filename}"
            gcs_hook.upload(
                bucket_name="data-lake-prod",
                object_name=gcs_path,
                data=buf.getvalue(),
                mime_type="text/csv",
            )
            uploaded.append(gcs_path)
            print(f"  Uploaded: {filename} → gs://data-lake-prod/{gcs_path}")
```

```python
    return uploaded   # XCom push — list of GCS paths for downstream tasks

# ── Scenario 2: Poll FTP for new files (custom sensor pattern) ──────────────
def check_ftp_files_arrived(ds: str, **context) -> bool:
    """
    Sensor-style check: return True only when expected file exists on FTP.
    Wrap this in a PythonSensor for automatic retry/wait behaviour.
    """
    hook = FTPHook(ftp_conn_id="partner_ftp")
    ds_nodash = ds.replace("-", "")
    expected  = f"/outbound/orders/{ds_nodash}/ORDERS_{ds_nodash}_COMPLETE.csv"

    try:
        with hook.get_conn() as ftp:
            ftp.size(expected)   # Raises ftplib.error_perm if not found
            return True
    except Exception:
        return False

# ── Scenario 3: Upload processed results back to partner FTP ────────────────
def deliver_results_via_ftp(local_path: str, **context) -> None:
    """
    After processing, upload the results file back to the partner's
    FTP inbox — common in B2B and EDI workflows.
    """
    hook     = FTPHook(ftp_conn_id="partner_ftp")
    filename = os.path.basename(local_path)

    with hook.get_conn() as ftp:
        ftp.cwd("/inbox/processed/")
        with open(local_path, "rb") as f:
            ftp.storbinary(f"STOR {filename}", f)
        print(f"Delivered: {filename} to partner FTP /inbox/processed/")
```

SFTP / SSH File Transfer

3.8 SFTPHook

```python
from airflow.providers.sftp.hooks.sftp import SFTPHook
```

Provides secure file transfer over SSH using SFTP (SSH File Transfer Protocol) via the paramiko library. Supports both password and private key authentication — keys are stored in the Connection's extra JSON field or as a file path. Unlike FTPHook, all transfers are encrypted end-to-end. Use SFTPHook for any file exchange that requires security compliance — server certificates, financial data, healthcare records (HIPAA), or data center-to-cloud transfers.

PYTHON

```python
from airflow.providers.sftp.hooks.sftp import SFTPHook
from airflow.providers.google.cloud.hooks.gcs import GCSHook
import io

# Connection 'sftp_prod' stores:
#   host: sftp.bank.com, login: ops-user, port: 22
#   extra: {"key_file": "/opt/airflow/keys/sftp_rsa", "no_host_key_check": false}

# ── Scenario 1: Download encrypted statement files from bank SFTP ──────────
def download_bank_statements(ds: str, **context) -> list:
    """
    Download daily bank statement files (PCI-DSS compliance requires SFTP).
    Files are PGP-encrypted on the server — we download and decrypt in memory.
```

```python
    """
    sftp_hook = SFTPHook(ssh_conn_id="sftp_bank_prod")
    gcs_hook  = GCSHook(gcp_conn_id="google_cloud_default")
    ds_nodash = ds.replace("-", "")

    remote_files = sftp_hook.list_directory(f"/statements/{ds_nodash}/")
    downloaded   = []

    for filename in remote_files:
        if not filename.endswith(".pgp"):
            continue

        # Retrieve file bytes directly into memory
        remote_path = f"/statements/{ds_nodash}/{filename}"
        file_bytes  = sftp_hook.retrieve_file(remote_path, None)  # returns BytesIO

        # Stage encrypted file to GCS for PGP decryption step
        gcs_path = f"raw/bank-sftp/{ds}/{filename}"
        gcs_hook.upload("secure-data-lake", gcs_path, data=file_bytes.read())
        downloaded.append(gcs_path)
        print(f"Downloaded (encrypted): {filename}")

    return downloaded

# ── Scenario 2: SFTPHook as the basis for a custom SFTP sensor ──────────────
def sftp_file_exists(remote_path: str, **context) -> bool:
    """
    Check if a file exists on the SFTP server.
    Use inside a PythonSensor for polling behaviour.
    """
    hook = SFTPHook(ssh_conn_id="sftp_prod")
    try:
        hook.get_file_by_pattern(
            path=remote_path,
            fnmatch_pattern="*",
        )
        return True
    except Exception:
        return False

# ── Scenario 3: Upload reports to a payment processor via SFTP ──────────────
def upload_reconciliation_report(gcs_path: str, **context) -> None:
    """
    Pull processed report from GCS and deliver via SFTP to payment processor.
    Follows PCI-DSS requirements for data at rest and in transit.
    """
    from airflow.providers.google.cloud.hooks.gcs import GCSHook
    import tempfile, os

    gcs_hook  = GCSHook(gcp_conn_id="google_cloud_default")
    sftp_hook = SFTPHook(ssh_conn_id="sftp_payment_processor")
    ds        = context["ds"]

    # Download from GCS to temp file
    with tempfile.NamedTemporaryFile(suffix=".csv", delete=False) as tmp:
        tmp_path = tmp.name
        gcs_hook.download("secure-data-lake", gcs_path, filename=tmp_path)

    try:
        remote_filename = f"RECON_{ds.replace('-','')}.csv"
        sftp_hook.store_file(
            remote_full_path=f"/incoming/{remote_filename}",
```

```python
            local_full_path=tmp_path,
        )
        print(f"Delivered reconciliation report: {remote_filename}")
    finally:
        os.unlink(tmp_path)   # Always clean up temp file
```

SSH Remote Execution

3.9 SSHHook

```python
from airflow.providers.ssh.hooks.ssh import SSHHook
```

> *Establishes an SSH connection to a remote host using paramiko and allows you to execute arbitrary shell commands on that remote machine from within an Airflow task. Supports both password and private key authentication. Connection type is ssh . Use SSHHook when you need to run commands on servers that are not Airflow workers — triggering scripts on database servers, running commands on on-premise Spark clusters, or managing remote infrastructure as part of a pipeline.*

PYTHON

```python
from airflow.providers.ssh.hooks.ssh import SSHHook

# Connection 'ssh_dw_server' stores:
#   host: dw-server-01.internal, login: airflow-agent, port: 22
#   extra: {"key_file": "/opt/airflow/keys/id_rsa", "timeout": 30, "compress": true}

# ── Scenario 1: Run a database export on a remote DB server ──────────────────
def export_db_snapshot(ds: str, **context) -> str:
    """
    SSH into the on-premise Postgres server and run pg_dump.
    The server is not directly accessible from the internet — only via SSH.
    """
    hook   = SSHHook(ssh_conn_id="ssh_dw_server", timeout=600)
    client = hook.get_conn()

    ds_nodash  = ds.replace("-", "")
    dump_path  = f"/opt/exports/orders_{ds_nodash}.sql.gz"
    command    = (
        f"pg_dump -h localhost -U readonly_user -d orders_db "
        f"--table public.transactions "
        f"-Fc --compress=9 -f {dump_path} "
        f"&& echo 'EXPORT_OK:{dump_path}'"
    )

    stdin, stdout, stderr = client.exec_command(command, timeout=580)
    exit_code = stdout.channel.recv_exit_status()

    output = stdout.read().decode("utf-8").strip()
    errors = stderr.read().decode("utf-8").strip()

    client.close()

    if exit_code != 0:
        raise RuntimeError(f"pg_dump failed (exit {exit_code}): {errors}")

    print(f"DB export complete: {dump_path}")
    return dump_path   # Push to XCom for next task (SFTP upload)

# ── Scenario 2: Trigger a script on a legacy Hadoop cluster ──────────────────
def run_hive_job(date: str, **context) -> dict:
    """
    SSH into the on-premise Hive server and run a HiveQL script.
```

```python
    Used for legacy Hadoop clusters not yet migrated to cloud.
    """
    hook   = SSHHook(ssh_conn_id="ssh_hadoop_namenode", timeout=3600)
    client = hook.get_conn()

    # Run the Hive script remotely and capture output
    command = f"hive -f /opt/scripts/daily_aggregation.hql -hivevar run_date={date}"
    stdin, stdout, stderr = client.exec_command(command, timeout=3590)

    # Stream output line-by-line for live logging
    for line in iter(stdout.readline, ""):
        print(f"[Hive] {line.rstrip()}")

    exit_code = stdout.channel.recv_exit_status()
    client.close()

    if exit_code != 0:
        raise RuntimeError(f"Hive job failed: {stderr.read().decode()}")

    return {"status": "success", "date": date}

# ── Scenario 3: Use SSHHook for port forwarding (tunnel to internal DB) ──────
def query_via_ssh_tunnel(ds: str, **context) -> list:
    """
    Open an SSH tunnel to reach a database that is only accessible
    from the bastion host — common in VPC/private subnet deployments.
    """
    import pandas as pd
    import sqlalchemy

    hook   = SSHHook(ssh_conn_id="ssh_bastion_host")
    tunnel = hook.get_tunnel(
        remote_host="internal-db.private",
        remote_port=5432,
        local_port=15432,    # bind tunnel to localhost:15432
    )

    with tunnel:
        engine = sqlalchemy.create_engine(
            f"postgresql+psycopg2://readonly:pass@localhost:15432/analytics"
        )
        df = pd.read_sql(
            f"SELECT * FROM orders WHERE order_date = '{ds}'",
            engine,
        )

    return df.to_dict(orient="records")
```

SMTP / Email 3.10 EmailHook

```python
from airflow.providers.smtp.hooks.smtp import SmtpHook
```

> *Sends emails via SMTP using server credentials stored in an Airflow Connection. Supports HTML and plain-text bodies, file attachments, CC/BCC recipients, and custom headers. The Connection stores SMTP host, port, login, password, and SSL settings. In Airflow 2.x, the hook was available as EmailHook from airflow.hooks.email ; from Airflow 2.4+ it is provided by the SMTP provider as SmtpHook . Both are functionally identical. Used by EmailOperator internally.*

PYTHON

```python
from airflow.providers.smtp.hooks.smtp import SmtpHook
# Legacy alias (Airflow 2.x): from airflow.hooks.email import EmailHook

# Connection 'smtp_prod' stores:
#   host: smtp.gmail.com (or smtp.sendgrid.net etc.), port: 587
#   login: alerts@company.com, password: app-password
#   extra: {"use_tls": true}

# ── Scenario 1: Send rich HTML pipeline summary to stakeholders ──────────────
def send_pipeline_summary(ds: str, stats: dict, **context) -> None:
    """
    After ETL completes, email a formatted HTML summary to data owners.
    Dynamically builds recipient list and injects pipeline metrics.
    """
    recipients = [
        "analytics-lead@company.com",
        "data-engineering@company.com",
    ]

    # Build a professional HTML email body
    html_content = f"""

    📊 Daily ETL Pipeline Report
    Pipeline {context['dag'].dag_id}
       completed successfully for {ds}.



        MetricValue

      Rows Extracted{stats.get('rows_extracted', 0):,}
      Rows Loaded{stats.get('rows_loaded', 0):,}
      Quality Score{stats.get('quality_pct', 0):.1f}%
      Duration{stats.get('duration_min', 0):.1f} min
      Run ID{context['run_id']}



      Sent automatically by Airflow. To unsubscribe, contact data-engineering@company.com


    """

    with SmtpHook(smtp_conn_id="smtp_prod") as hook:
        hook.send_email_smtp(
            to=recipients,
            subject=f"[Airflow] ETL Pipeline Complete — {ds}",
            html_content=html_content,
            from_email="airflow-alerts@company.com",
        )
    print(f"Pipeline summary sent to {len(recipients)} recipients")

# ── Scenario 2: Attach a CSV report as a file attachment ──────────────────
def email_dq_report_with_attachment(ds: str, **context) -> None:
    """
    Generate a data quality CSV report and send it as an email attachment.
    """
    import csv, io

    # Build CSV report in memory
    rows = [
```

```python
        ["Table", "Row Count", "Null PKs", "Duplicates", "Status"],
        ["fact_orders", 120_483, 0, 0, "PASS"],
        ["dim_customers", 98_721, 0, 2, "WARN"],
        ["agg_daily_revenue", 365, 0, 0, "PASS"],
    ]
    buf = io.StringIO()
    csv.writer(buf).writerows(rows)
    csv_bytes = buf.getvalue().encode("utf-8")

    with SmtpHook(smtp_conn_id="smtp_prod") as hook:
        hook.send_email_smtp(
            to=["data-quality-team@company.com"],
            subject=f"[DQ Report] Data Quality Results — {ds}",
            html_content=f"Please find the data quality report for {ds} attached.",
            files=[
                {
                    "filename": f"dq_report_{ds}.csv",
                    "content":  csv_bytes,
                    "mime_type": "text/csv",
                }
            ],
        )

# ── Scenario 3: on_failure_callback — rich failure alert ──────────────────
def rich_failure_alert(context) -> None:
    """
    Use EmailHook inside on_failure_callback for HTML failure emails.
    More powerful than the built-in email_on_failure=True simple text email.
    """
    dag_id  = context["dag"].dag_id
    task_id = context["task_instance"].task_id
    exc     = str(context.get("exception", "Unknown error"))[:500]
    log_url = context["task_instance"].log_url

    html = f"""

    ❌ Airflow Task Failed

      DAG{dag_id}
      Task{task_id}
      Error{exc}

    View Task Logs →

    """

    with SmtpHook(smtp_conn_id="smtp_prod") as hook:
        hook.send_email_smtp(
            to=["on-call@company.com"],
            subject=f"[ALERT] Airflow Failure: {dag_id}.{task_id}",
            html_content=html,
            from_email="airflow-alerts@company.com",
        )

# Attach to a DAG:
DEFAULT_ARGS = {"on_failure_callback": rich_failure_alert}
```